

LoReflection 实验框架

实验要证明

代码块

- 1 现有方法多数是 one-shot generation / one-shot editing;
- 2 LoReflection 通过 observe-review-repair-verify 的闭环反思机制,
- 3 能够把初始生成结果进一步提升为更高质量的最终 3D scene。

布局生成结果对比

Table 1

Type	Method	Scene Quality			Semantic Compliance	
		CountF1 ↑	FID ↓	KID ↓	Sem.SR ↑	OOB ↓
A2L	ATISS					
	MiDiffusion					
	SemLayoutDiff					
	Ours-Initial (Arch.)					
	Ours-Final (Arch.)					
T2L	DirectLayout					
TA2L	ReSpace					
	Ours-Initial (Text)					
	Ours-Final (Text)					

图类似下面

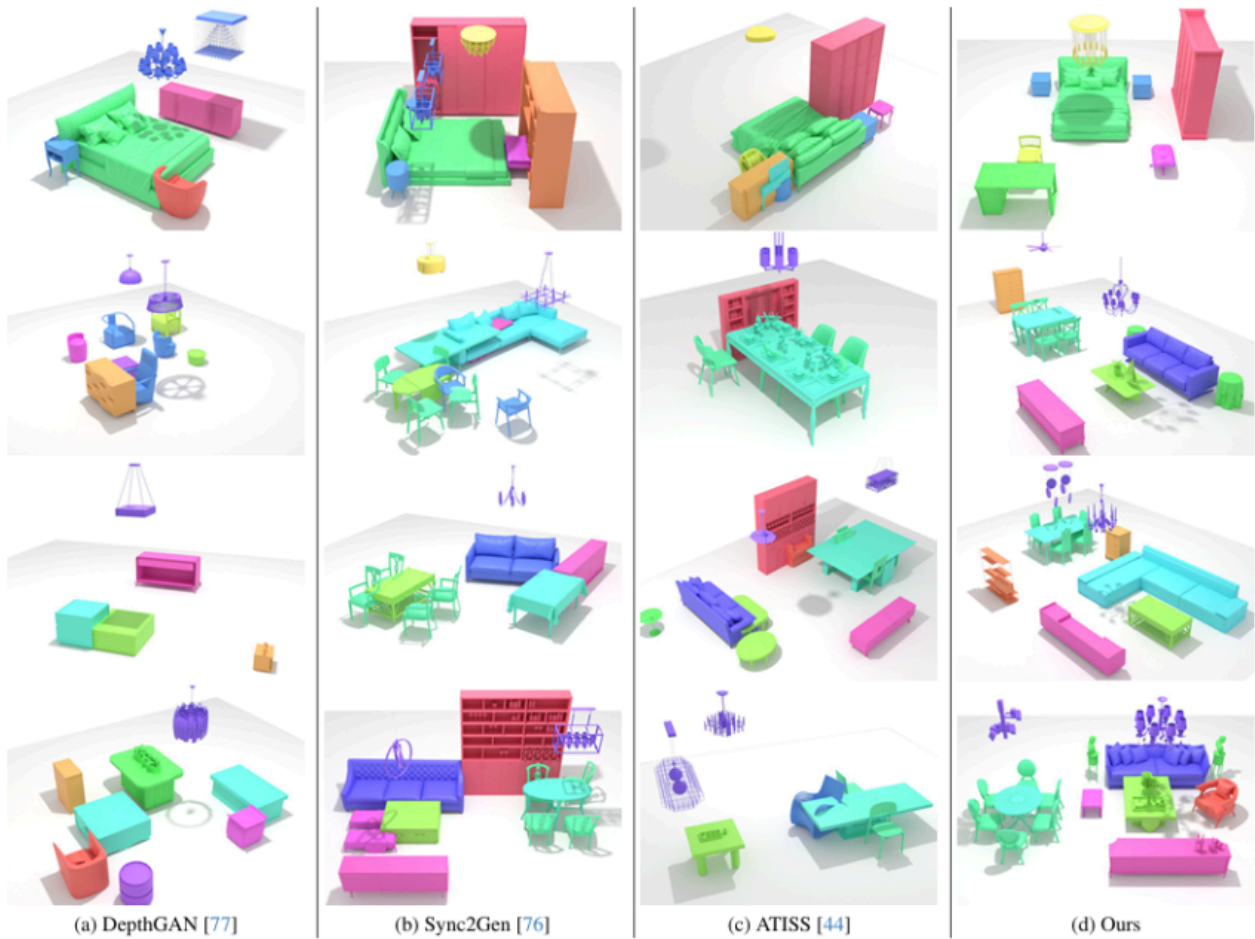


Figure 4. **Unconditional scene synthesis.** We compare our method with the state-of-the-art by generating from random noises, where our results present higher diversity and better plausibility with fewer penetration issues and more symmetric pairs.

编辑结果对比

Table 2: Language-guided Layout Editing Quality

Method	Editing-specific			Semantic Compliance	Geometric Validity
	Edit Success $\uparrow$	Target Loc. $\uparrow$	NT-Pres. $\uparrow$	Func.Rel SR $\uparrow$	Edit-induced Hard Error $\downarrow$
DiffuScene-N $\dagger$					
SceneEditor-N $\dagger$					
EditRoom					
DirectLayout Re-generation					
Ours-Final					

指标对应关系

Category	Metric	Meaning
Editing-specific	Edit Success ↑	编辑命令是否被成功执行，包括 translate、rotate、scale、replace、add、remove 六类操作。
Editing-specific	Target Loc. ↑	编辑是否作用在正确目标对象上，避免移动或修改错误家具。
Editing-specific	NT-Pres. ↑	protected non-target objects 是否保持稳定，检查类别、位置、尺寸、朝向和 footprint IoU。
Semantic Compliance	Func.Rel SR ↑	编辑后布局是否仍满足房间功能关系，例如 sofa facing TV、nightstand beside bed、desk near window。
Geometric Validity	Edit-induced Hard Error ↓	编辑后相对于 source layout 新增的硬错误，包括 new collision、new OOB、new door/window blocking。
Practical Usability	Practical SR ↑	编辑后布局是否仍具备真实使用可用性，包括功能距离、开合净空和通行净空。

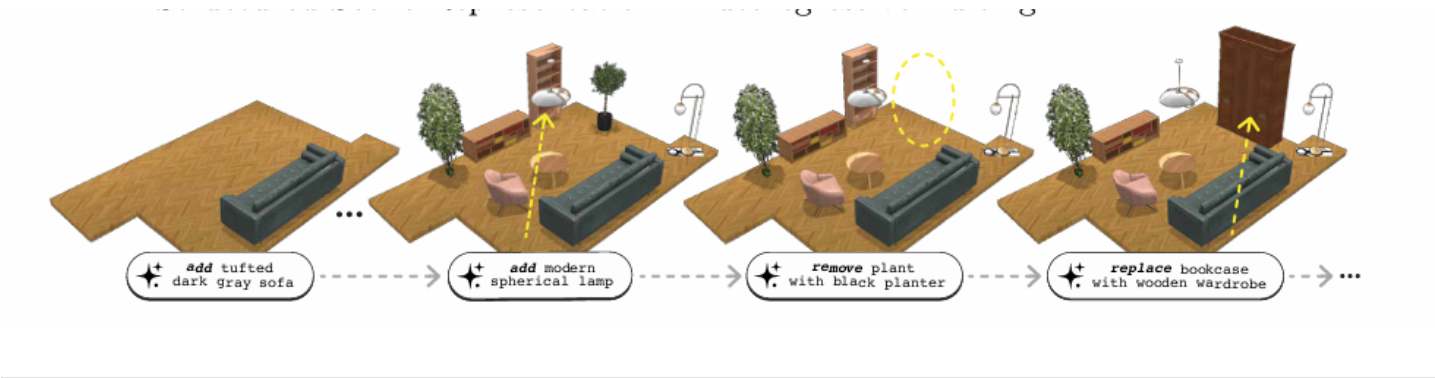
表注

† DiffuScene-N 和 SceneEditor-N 仅在能够按照相同 EditRoom-style protocol 复现时纳入主表。

DirectLayout Re-generation 不是严格的 editing method，而是 global regeneration baseline，用于检验全局重生成是否会破坏 non-target objects 或引入新的 hard errors。

主表报告 single-operation editing 的平均结果；per-edit-type breakdown 放入附录。

图的话类似下面这样



消融实验

- 1. 闭环反思机制是否有效；
- 2. LoState / LoReview / LoRAM 是否提供了可靠的问题归因；

Table 3: Ablation on Reflection, Review, and Repair Components

Method	Ablated Component	Target Repair	Semantic Compliance	Geometric Validity		Practical Usability
		Target Res. ↑	Rel. Red. ↑	Hard Red. ↑	New Hard ↓	Practical SR ↑
Full LoReflection	None					
w/o Reflection Loop	observe-review-repair-verify loop					
One-shot Repair	multi-round re-observe / re-repair / rollback					
LoState → Free Text	structured Goal / Observed LoState					
w/o Track A	programmatic geometric review					
w/o Track B	VLM semantic review					
w/o LoRAM	relation-aware slot-instance attribution					
w/o AcceptanceController	verify / accept / rollback					

Metric Meaning

Category	Metric	Meaning
Target Repair	Target Res. ↑	目标问题是否被成功修复，例如 missing object 被补全、wrong relation 被修正、door blocking 被解除。
Semantic Compliance	Rel. Red. ↑	修复前后关系错误的减少比例，包括 not beside、not near、wrong facing、not against wall 等。
Geometric Validity	Hard Red. ↑	修复前后硬几何错误的减少比例，包括 collision、OOB、door / window blocking。
Geometric Validity	New Hard ↓	修复过程中是否引入新的硬错误，例如新增 collision、OOB 或 door / window blocking。
Practical Usability	Practical SR ↑	修复后布局是否具备真实使用可用性，包括功能距离、开合净空和通行净空。
Preservation	NT-Pres. ↑	非目标对象是否保持稳定，包括 category、center position、size、orientation 和 footprint IoU。

Table Purpose

这张表用于回答 LoReflection 的核心组件是否必要：

- **w/o Reflection Loop** 验证闭环反思机制是否带来整体提升。
- **One-shot Repair** 验证单轮修复是否不如多轮 re-observe / repair / accept。

- **LoState → Free Text** 验证结构化状态表示是否比自由文本更稳定。
- **w/o Track A** 验证程序化几何审查对 hard violation reduction 和 new hard error 控制的贡献。
- **w/o Track B** 验证 VLM 语义审查对 relation repair 和 semantic compliance 的贡献。
- **w/o LoRAM** 验证关系感知归因是否有助于正确定位目标对象和错误类型。
- **w/o AcceptanceController** 验证验收 / 回滚机制是否能减少有害修复和非目标破坏。

method	对象级		关系级		建筑级		编辑保持性诊断	
	指标1	指标2	指标1	指标2	指标1	指标2	指标1	指标2
闭环反思机制是否有效；								
LoState / LoReview / LoRAM 是否提供了可靠的问题归因；								
ours								

User Study

进行两组匿名 pairwise human evaluation，用于补充自动指标难以完全覆盖的主观质量判断。所有样本均采用随机左右顺序、统一渲染协议，并隐藏方法名称。

Table 6: Human Evaluation on Text-conditioned Synthesis

该用户研究对应 **Table 1 的 Text-conditioned Synthesis 部分**，用于验证 Ours-Final (Text) 是否比 direct text-to-layout / structured text-driven baselines 更符合文本描述，并生成更合理、更可用的室内布局。

Comparison	Text Match ↑	Fewer Geom. Errors ↑	Practical Usability ↑	Overall ↑
Ours-Final vs DirectLayout				
Ours-Final vs ReSpace				
Ours-Final vs Ours-Initial				

Table 7: Human Evaluation on Language-guided Editing

该用户研究对应 **Table 2: Language-guided Layout Editing Quality**，用于验证 LoReflection 在 language-guided editing 中是否不仅能完成编辑命令，还能保持 non-target objects，并减少 edit-induced visible errors。

Comparison	Edit Sat. ↑	Target Correct ↑	NT-Pres. ↑	Fewer Errors ↑	Overall ↑
Ours-Final vs EditRoom					
Ours-Final vs DirectLayout Re-generation					
Ours-Final vs Ours-Initial / One-shot Repair					

Table 4. User rankings for the three methods on 40 samples.

Method	AR↓	Rank Distribution		
		1st%	2nd%	3rd%
DIFFUSCENE [39]	2.21	11.11	57.22	31.67
MIDIFFUSION [16]	2.55	8.47	28.33	63.19
SEMLAYOUTDIFF (Ours)	1.25	80.42	14.44	5.14

Table 5. Fine-grained user evaluation. Users were asked to assess errors found in generated scenes.

Method	Intersection		Blocked Door↓	Implausible	
	Arch↓	Obj↓		Layout↓	Orientation↓
DIFFUSCENE [39]	89.38	38.12	41.25	45.00	25.62
MIDIFFUSION [16]	80.62	60.00	27.50	78.12	55.62
SEMLAYOUTDIFF (Ours)	20.00	25.00	22.50	11.88	24.38

Computational Cost Analysis

- 迭代反思机制（TOTE 循环）虽然提高了质量，但带来了巨大的时间延迟和 API Token 成本。这在实际工业应用中是否具有可行性？

实验目的

尽管基于 `Observe-Review-Repair-Verify` 的闭环反思空间（LoReflection）显著提升了室内布局的生成质量，但多轮 VLM 推理与局部潜空间重绘（Latent Replacement）必然会带来额外的计算延时与 API Token 消耗。本实验旨在**定量评估系统的效率-质量权衡（Efficiency-Quality Trade-off）**，并有力反驳审稿人对“闭环迭代导致计算成本无限膨胀”的顾虑。我们将证明，LoReflection 具有极佳的**早停经济性（Early-Stopping Viability）与单调收敛性**，通常仅需 2-3 轮迭代即可在极低成本下消除 90% 以上的硬性物理违规。

对比方法

为了全面评估开销的多维坐标，我们设置了纵向时序变体与横向范式基线的双重对比：

- 纵向时序变体（Internal Steps）**：`Ours (t=0)`（即初始未反思状态 `Ours-Initial`）、`Ours (t=1)`、`Ours (t=2)`、`Ours (t=3)` 以及最大迭代上界 `Ours (T_max=5)`。
- 开放场景文本规划基线（Open-loop LLM）**：`LayoutGPT`。作为“单次开闭环推理（One-shot Open-loop）”的代表，评估其零反思但极低的时间成本。
- 连续可微几何优化基线（Optimization-based）**：`LayoutVLM`。作为“基于梯度的连续优化范式”代表，评估其运行完整优化流（500 steps）所需的绝对时间。
- 本实验主要进行**自身迭代轮数 $t = 0, 1, 2, 3, \dots, T_{\{max\}}$ 的纵向对比**，并引入连续可微优化模型 `LayoutVLM` 作为时间开销对照（`LayoutVLM` 优化 500 个 steps 所需的时间）。

评估指标

为了使成本评估具备严密的统计学意义，我们定义了以下四项核心量化指标：

- Avg. Wall-clock Time (s) ↓**：从输入原始户型/文本指令，到系统输出最终验收布局的平均绝对端到端耗时（秒）。
- Cumulative Token Count ↓**：系统在多轮交互中累积消耗的底层大模型 API Tokens（区分 Prompt Token 与 Completion Token）。
- Hard Violation Drop Rate (%) ↑**：相比于初始状态（ $t=0$ ），每一轮迭代所消除的硬性违规（`Collision` 与 `OOB`）的比例。
- Success Convergence Rate (%) ↑**：在指定迭代步数 t 内，完美通过验证器（Verifier）检查、实现“硬性冲突清零”的场景样本百分比。

Table 8：多范式计算开销与质量增益定量对比表

目的：该表用于向审稿人直观展示：虽然我们用了迭代，但我们在极少轮数下的时间远比连续优化模型 `LayoutVLM` 节省，且质量远超 `LayoutGPT`。

Method / Variant	Iter. Steps (t)	Avg. Time (s) ↓	Prompt Tokens ↓	Completion Tokens ↓	Hard Violation Drop ↑	Convergence Rate ↑
LayoutGPT	1 (One-shot)					
LayoutVLM	500 (Steps)					
Ours (Initial)	0					
Ours (Iter 1)	1					
Ours (Iter 2)	2					
Ours (Iter 3)	3					
Ours (Final)	<5					

Figure3：效率-质量帕累托前沿与成本单调收敛曲线图
(Pareto Frontier & Convergence Analysis)

图表设计：采用双轴折线图。横轴（X轴）统一为系统迭代轮数 t （从 0 到 5）。左侧纵轴（Y1，黑色/蓝色曲线）代表全屋综合硬性冲突率（`Collision Rate + 00B Rate`）；右侧纵轴（Y2，红色曲线）代表端到端累积 `Wall-clock Time` 或 `Token` 开销。

预期趋势与学术结论：

- a. **对数骤降趋势**：物理违规率曲线在 $t = 0 \rightarrow 1 \rightarrow 2$ 时呈现急剧的对数式下跌，在 $t = 2$ 时实现拐点并迅速拉平。
- b. **线性成本上升**：时间与 Token 开销随 t 呈现平缓的线性上升趋势。
- c. **帕累托最优（Pareto Optimum）**：双轴曲线的交汇拐点完美锁定在 $t = 2$ 。这强有力地证明了系统拥有极佳的**早停边界**。系统绝非无底洞式的死循环，在实际工业落地中，通常只需配置 2 轮反思，即可在极低计算预算下撬动最优的布局质量。